

Controllable Graph Generation with Diffusion Models via Inference-Time Tree Search Guidance

Jiachi Zhao^{*†}
University of Notre Dame
Notre Dame, Indiana, USA
jiachizhao2@gmail.com

Zehong Wang^{*‡}
University of Notre Dame
Notre Dame, Indiana, USA
zwang43@nd.edu

Yamei Liao
Independent Researcher
New York City, USA
yameiliao.dt@gmail.com

Chuxu Zhang
University of Connecticut
Storrs, Connecticut, USA
chuxu.zhang@uconn.edu

Yanfang Ye[‡]
University of Notre Dame
Notre Dame, Indiana, USA
yye7@nd.edu

Abstract

Graph generation is a fundamental problem in graph learning with broad applications across Web-scale systems, knowledge graphs, and scientific domains such as drug and material discovery. Recent approaches leverage diffusion models for step-by-step generation, yet unconditional diffusion offers little control over desired properties, often leading to unstable quality and difficulty in incorporating new objectives. Inference-time guidance methods mitigate these issues by adjusting the sampling process without retraining, but they remain inherently local, heuristic, and limited in controllability. To overcome these limitations, we propose TreeDiff, a Monte Carlo Tree Search (MCTS) guided dual-space diffusion framework for controllable graph generation. TreeDiff is a plug-and-play inference-time method that expands the search space while keeping computation tractable. Specifically, TreeDiff introduces three key designs to make it practical and scalable: (1) a macro-step expansion strategy that groups multiple denoising updates into a single transition, reducing tree depth and enabling long-horizon exploration; (2) a dual-space denoising mechanism that couples efficient latent-space denoising with lightweight discrete correction in graph space, ensuring both scalability and structural fidelity; and (3) a dual-space verifier that predicts long-term rewards from partially denoised graphs, enabling early value estimation and removing the need for full rollouts. Extensive experiments on 2D and 3D molecular generation benchmarks, under both unconditional and conditional settings, demonstrate that TreeDiff achieves state-of-the-art performance. Notably, TreeDiff exhibits favorable inference-time scaling: it continues to improve with additional computation, while existing inference-time methods plateau early under limited resources.

^{*}Equal Contribution.

[†]Visiting Student at the University of Notre Dame.

[‡]Corresponding Authors.

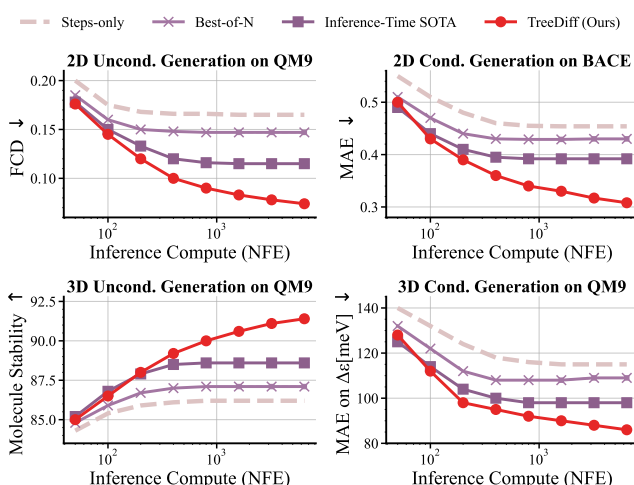


Figure 1: Inference-time scaling behavior on four generation benchmarks. TreeDiff exhibits consistent gains with additional inference computation, whereas existing approaches—including standard diffusion [34], Best-of-N [46], and state-of-the-art inference-time guidance [40]—saturate quickly, highlighting the scalability of our approach.

CCS Concepts

• Information systems → Data mining; • Applied computing → Computational biology; • Computing methodologies → Machine learning approaches.

Keywords

Graph Generation, Diffusion Models, Monte Carlo Tree Search

ACM Reference Format:

Jiachi Zhao, Zehong Wang, Yamei Liao, Chuxu Zhang, and Yanfang Ye. 2026. Controllable Graph Generation with Diffusion Models via Inference-Time Tree Search Guidance. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792489>

1 Introduction

Graphs are powerful structures for modeling complex relationships among entities [23, 38, 71, 75, 78] and form the backbone of numerous Web-related systems [35, 93], such as knowledge



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792489>

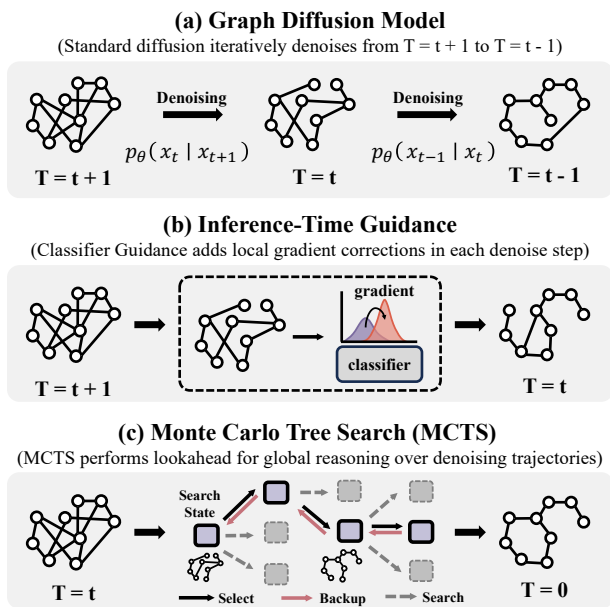


Figure 2: Comparison among diffusion-based graph generation approaches. (a) Standard graph diffusion iteratively denoises each step without correction. (b) Inference-time guidance introduces local corrections via classifier gradients but remains short-sighted. (c) Monte Carlo Tree Search (MCTS) enables structured lookahead over denoising trajectories, allowing for globally balanced exploration and exploitation.

graphs, recommender networks, and online interaction graphs. The ability to generate and manipulate such graphs under domain-specific constraints has become a fundamental problem in graph learning [3, 43, 62, 64, 73]. For example, in molecular and material design [43]—where molecules can be viewed as structured graphs—generation models aim to discover new candidates that satisfy specific chemical or physical properties. Similarly, in broader Web contexts, graph generation underpins applications such as link prediction [23], knowledge graph completion [7], and the simulation of evolving information networks [49]. Despite these diverse applications, generating graphs that are valid, diverse, and aligned with complex objectives remains a significant algorithmic challenge.

Various generative models have been explored for graph generation [4, 31, 32, 62, 92], among which diffusion models [27, 34, 43, 73, 86] have recently gained prominence due to their ability to generate diverse, high-quality samples. Specifically, a diffusion model [25] learns the reverse denoising process during training to approximate the data distribution. During inference, it starts from noise and iteratively denoises to recover valid samples, ensuring consistency between training and inference. Despite the success of diffusion models in graph generation, they face two major limitations that hinder their practicality. First, controllability is limited [54, 56]: when trained without explicit conditions or constraints, diffusion models primarily approximate the unconditional data distribution, lacking a principled way to enforce target properties during sampling. Second, generation can be unstable [80]: the stochasticity

of noise sampling often leads to invalid or low-quality outputs, reflecting the inherent fragility of the denoising process. These issues are especially severe in graph generation, where tasks like drug discovery demand molecules with domain-specific attributes (e.g., high QED, low SAS), yet such properties are rarely optimized during training. Moreover, graphs are far less tolerant to perturbations than images—even minor structural changes can drastically alter graph semantics [77, 79].

To this end, recent works have proposed inference-time guidance methods [14, 40, 46], which steer the sampling process at test time without altering training. Specifically, these methods refine sampling under given conditions and prune or adjust invalid steps when necessary, thereby enabling more stable and controllable generation. For instance, Ma et al. [46] maintain a pool of candidate noises and perform zero-order search to expand promising branches during inference for better generation quality. Similarly, Dorna et al. [14] leverage an auxiliary classifier as a reward model to estimate the quality of intermediate states and inject gradient signals for controllable graph generation. Despite their effectiveness, existing approaches remain fundamentally local: they make decisions based on step-level feedback, greedily correcting or discarding samples according to the current state without anticipating long-term consequences. This reliance on local information limits their ability to plan across long denoising trajectories or to satisfy complex, multi-objective constraints, leaving controllability and stability still restricted [88]. Empirically, this limitation manifests in poor inference-time scaling: as illustrated in Figure 1, increasing computation does not improve—and can even harm—the performance of existing approaches, underscoring their inability to benefit from additional computing resources.

To overcome the limitation, we propose using Monte Carlo Tree Search (MCTS) [63] as a novel inference-time controller for graph generation. Unlike greedy stepwise heuristics, MCTS enables lookahead search, evaluates future trajectories, and integrates multiple objectives in a principled manner, offering a more stable and controllable generation process. However, directly applying MCTS to diffusion-based graph generation is non-trivial and introduces several unique challenges. (1) *Tree Depth Explosion*: Diffusion models typically require hundreds or even thousands of denoising steps, making one-step-per-node expansion computationally infeasible. This calls for mechanisms that can reduce the effective depth of the search while preserving the overall denoising trajectory. (2) *Ill-Defined Node Expansion Space*: Defining the expansion space is also problematic. Denoising directly in graph space ensures stability and validity but is extremely slow [73], while relying solely on embedding space is efficient but unstable [14], as small perturbations can lead to drastic structural changes. This highlights the need for strategies that balance efficiency with structural reliability during node expansion. (3) *Costly and Complex Evaluation*: MCTS requires reliable evaluation of future outcomes, yet in graph diffusion, full rollouts are prohibitively expensive: one must complete the entire denoising trajectory and compute molecular properties (e.g., QED, SAS), while also enforcing validity constraints. This makes it essential to design evaluation schemes that approximate long-term quality efficiently while pruning infeasible branches early.

To overcome these challenges, we introduce **TreeDiff**, a MCTS-guided dual-space diffusion framework for controllable graph generation. Specifically, TreeDiff incorporates three key designs: (1) a macro-step expansion strategy that reduces tree depth by grouping multiple denoising steps into one node, (2) a dual-space denoising mechanism that combines efficiency in the embedding space with stability in the graph space, and (3) a dual-space verifier that provides long-term value estimates to guide global search under complex validity constraints. Extensive experiments on 2D and 3D molecular benchmarks show that TreeDiff achieves state-of-the-art performance on both unconditional and conditional generation tasks. Notably, as shown in Figure 1, TreeDiff exhibits favorable inference-time scaling behavior—continuing to improve as more computation is allocated, whereas existing methods quickly plateau.

2 Related Work

Graph Generation. Graph generation [3, 68, 69] learns distributions over graph topology and attributes, enabling both unconditional and conditional synthesis. However, validity, property control, and efficiency remain difficult to achieve simultaneously [47, 85]. Prior work mainly falls into four families: autoregressive models allow rule injection but are order-sensitive and slow [41, 62, 90]; VAE-based methods support latent interpolation but impose size limits [32, 64]; GAN-based approaches provide one-shot fidelity but suffer from instability [12]; and flow-based models yield tractable likelihoods but face dequantization bias [45, 92]. More recently, diffusion models [25, 65, 66] have emerged as the dominant paradigm, offering high-quality graph generation.

Graph Diffusion Models. Diffusion models [20, 25, 51, 61] have recently been adapted to graphs by injecting noise into discrete topology and attributes and learning a reverse denoising process. Representative works include GDSS [34], which learns joint node & adjacency scores in a continuous-time SDE; DiGress [72], which applies discrete transformations over nodes and edges; GeoDiff [87], which performs SE(3)-equivariant diffusion for 3D conformations; and NVDiff [6], which denoises node embeddings for scalability. Extensions further explore conditional generation [21, 22, 31, 39, 42, 83] and efficient sampling [8, 44]. Despite these advances, controllability and stability remain open challenges: enforcing graph-level constraints consistently across multi-step denoising is still unreliable. Our proposed TreeDiff directly addresses these issues at inference time without retraining.

Inference-Time Guidance. For graph diffusion, inference-time guidance [70] offers a way to enforce validity and target properties without retraining, by steering the denoising trajectory at test time. Existing approaches can be grouped into three categories: (i) classifier guidance [9, 13, 14, 26, 37], which trains auxiliary models to score graph properties (e.g., QED/SA) and provide gradients or rewards; (ii) sampling-based methods [5, 40, 46, 50, 55, 82], which resample from candidate noises or trajectories and retain the best ones; and (iii) search-based methods [16, 24, 28, 30, 36, 40, 53, 67, 89], which keep a small frontier and apply heuristics or light backtracking. While these methods improve controllability, in graph domains they remain limited: feedback is often noisy, validity is fragile, and scaling candidates rarely yields consistent gains. Our proposed TreeDiff instead employs MCTS for global planning, integrating

property guidance with validity checks along full trajectories to achieve robust, controllable, and scalable inference.

3 Preliminary

Problem Definition. We study the problem of graph generation. Formally, let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{E})$ denote a graph, where $\mathcal{V}$ is the set of nodes with $|\mathcal{V}| = N$, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges with $|\mathcal{E}| = E$, and $\mathbf{X}$ and $\mathbf{E}$ represent the node and edge feature matrices, respectively. Each node $v \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_v \in \mathbb{R}^{d_n}$, and each edge $e \in \mathcal{E}$ is associated with a feature vector $\mathbf{e}_e \in \mathbb{R}^{d_e}$ (if edge attributes are available). Given a target distribution $\mathcal{P}_{\text{data}}$ over graphs, the goal is to learn a generative model $p_{\theta}(\mathcal{G})$ such that $p_{\theta}(\mathcal{G}) \approx \mathcal{P}_{\text{data}}$. That is, the model should generate graphs that are both structurally valid and semantically consistent with the observed data, while preserving diversity and faithfully capturing the underlying distribution.

Graph Diffusion Model. We adopt the diffusion framework in the context of graph generation. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{E})$, the forward process constructs a sequence of latent variables $\{\mathcal{G}_t\}_{t=0}^T$, where $\mathcal{G}_0 = \mathcal{G}$ and $\mathcal{G}_T$ approximates a noise distribution. At each step, noise is added according to a predefined Markov transition:

$$q(\mathcal{G}_t | \mathcal{G}_{t-1}) = \mathcal{N}\left(\sqrt{1 - \beta_t} \mathcal{G}_{t-1}, \beta_t \mathbf{I}\right), \quad (1)$$

where $\{\beta_t\}_{t=1}^T$ is a variance schedule. The reverse process aims to learn a parameterized distribution $p_{\theta}(\mathcal{G}_{t-1} | \mathcal{G}_t)$ that approximates the true posterior $q(\mathcal{G}_{t-1} | \mathcal{G}_t, \mathcal{G}_0)$. Sampling proceeds by iteratively denoising from Gaussian noise toward the target graph $\mathcal{G}_0$. The training objective is typically a reweighted denoising score matching loss:

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{t, \mathcal{G}_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(\mathcal{G}_t, t)\|^2], \quad (2)$$

where ϵ is Gaussian noise and ϵ_{θ} is a neural network that predicts the noise component. In graph generation, diffusion can be applied either directly in the graph space (adjacency and feature matrices) or in a continuous latent space learned by an encoder.

Monte Carlo Tree Search. The denoising process in diffusion models can be naturally viewed as a sequential decision problem, where each step refines the current state and local errors may accumulate into invalid or low-quality graphs. This motivates the use of search-based methods that can plan across long denoising trajectories. Among them, MCTS is a widely used algorithm for decision-making in large spaces under uncertainty. Formally, MCTS builds a search tree $\mathcal{T}$, where each node $s \in \mathcal{T}$ represents a state (e.g., a partially denoised graph), and edges correspond to actions $a \in \mathcal{A}(s)$ (e.g., candidate denoising steps). Each iteration of MCTS consists of four phases: (i) *Selection*, choosing a path from the root to a leaf according to an exploration–exploitation policy (e.g., choosing a partial denoising trajectory); (ii) *Expansion*, adding a new child node to represent an unexplored state (e.g., the next denoising step); (iii) *Simulation*, estimating the long-term outcome by rolling out a trajectory from the new node (e.g., sampling a complete graph), yielding a reward R ; and (iv) *Backpropagation*, propagating R upward to update the statistics of nodes along the path. By adaptively allocating computation to promising branches, MCTS provides lookahead planning, maintains multiple candidate trajectories, and integrates domain-specific objectives.

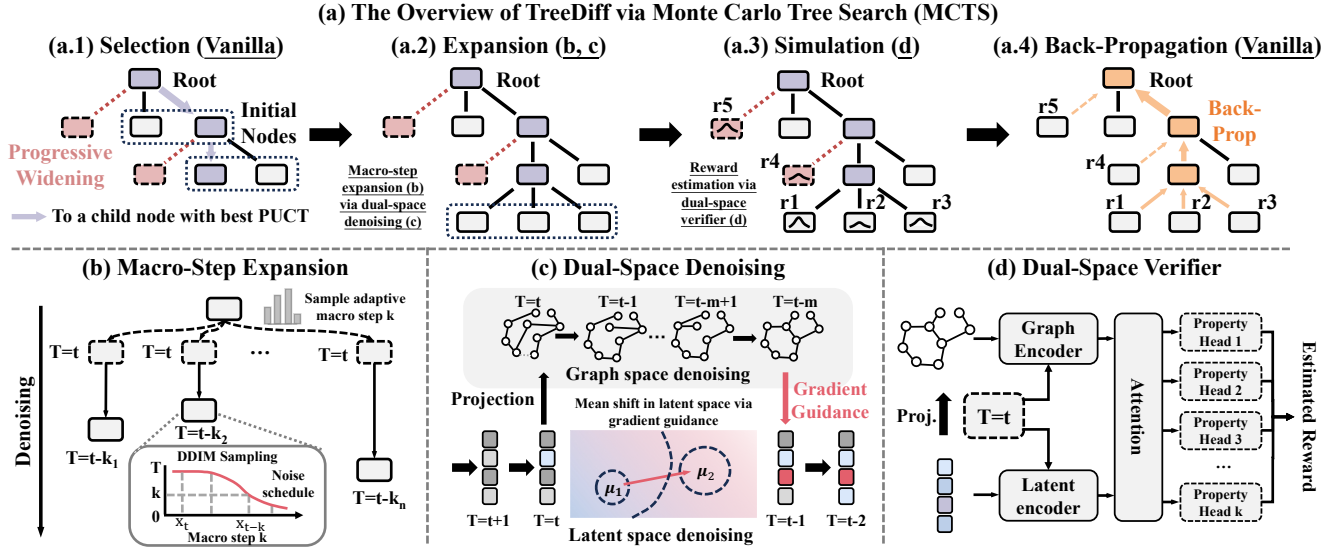


Figure 3: TreeDiff integrates MCTS with diffusion-based graph generation. (a) TreeDiff follows the standard MCTS pipeline but introduces new designs in the *Expansion* and *Simulation* phases. (b) *Macro-step expansion* reduces tree depth by grouping multiple denoising updates. (c) *Dual-space denoising* couples latent diffusion with discrete structural correction to ensure validity. (d) *Dual-space verifier* predicts long-term rewards from latent and graph embeddings, enabling rollout-free evaluation.

4 Methodology

We propose TreeDiff, an MCTS-guided diffusion framework for controllable and stable graph generation. TreeDiff formulates diffusion sampling as a sequential decision process and applies structured tree search to optimize denoising trajectories at inference time. Directly combining MCTS with diffusion is challenging due to three factors: (i) extremely long denoising horizons, (ii) the efficiency–stability trade-off between latent and graph spaces, and (iii) the high cost of full-trajectory evaluation. To overcome these challenges, TreeDiff introduces three components (Figure 3): (1) *macro-step expansion* to compress long trajectories into coarse transitions, (2) *dual-space denoising* to couple efficient latent updates with graph-level structural correction, and (3) a *dual-space verifier* to estimate long-term rewards without full rollouts. These designs enable adaptive inference-time computation and substantially improve generation controllability. Implementation details are provided in Appendix C.

4.1 Macro-Step Expansion

In MCTS, the *Expansion* phase determines how new nodes are added to the search tree. For diffusion-based generation, each reverse step only slightly refines the graph state, resulting in extremely deep trajectories. Expanding at every step would yield hundreds or thousands of near-duplicate layers, making exploration inefficient and long-horizon planning impractical. We redefine the expansion granularity using a *macro-step expansion* strategy that groups multiple denoising steps into a single transition. Instead of expanding from $\mathcal{G}_t$ to $\mathcal{G}_{t-1}$, TreeDiff directly expands to $\mathcal{G}_{t-k}$ for a horizon k , substantially reducing tree depth while preserving overall denoising dynamics. This converts expansion from step-wise local moves into hierarchical exploration over the denoising trajectory.

Formally, let $\mathcal{G}_t$ denote the intermediate graph at reverse step t . A single update is written as $\mathcal{G}_{t-1} = \Phi_\theta(\mathcal{G}_t, t; \xi_t)$, where Φ_θ is the learned reverse transition and $\xi_t \sim \mathcal{N}(0, \mathbf{I})$. A k -step macro transition is defined by composition:

$$\begin{aligned} \mathcal{G}_{t-k} &= \Phi_\theta^{(k)}(\mathcal{G}_t, t; \xi_{t-k+1:t}) \\ &\triangleq \Phi_\theta(\Phi_\theta^{(k-1)}(\mathcal{G}_t, t; \xi_{t-k+2:t}), t-k+1; \xi_{t-k+1}), \end{aligned} \quad (3)$$

with $\xi_{t-k+1:t} = \{\xi_{t-k+1}, \dots, \xi_t\}$. During *Expansion*, TreeDiff samples a small set of candidate macro-steps and inserts the resulting states as children of $\mathcal{G}_t$ in the search tree $\mathcal{T}$.

Adaptive Step Sampling. Rather than fixing k , TreeDiff adapts the adaptive expansion. Given remaining diffusion steps $T_{\text{rem}} = t$ and remaining tree depth D_{rem} , we set $k_{\text{base}} = \frac{T_{\text{rem}}}{D_{\text{rem}}}$, and sample

$$k \sim \mathcal{N}(k_{\text{base}}, (\sigma_k k_{\text{base}})^2), \quad 1 \leq k \leq T_{\text{rem}}, \quad (4)$$

where $\sigma_k = 0.1$. This stochastic scheduling enables both coarse long-range moves and fine local refinement, improving the exploration–exploitation trade-off. When $T_{\text{rem}} < k_{\text{base}}$, the process directly denoises to $\mathcal{G}_0$.

4.2 Dual-Space Denoising

Macro-step expansion enables efficient exploration but raises a key question for MCTS: *which representation should be used for selection and expansion?* Each node can be represented in either latent or graph space, yet both have limitations. Latent-space search provides smooth transitions for UCB [1] or PUCT [63], but decoded trajectories may become invalid. Graph-space search preserves structural validity and enables property evaluation, but introduces combinatorial branching and discontinuous transitions, leading to unstable selection. This mismatch hinders effective long-horizon planning.

To address it, TreeDiff adopts *dual-space denoising*, coupling continuous latent diffusion with lightweight graph-space correction.

Specifically, dual-space denoising governs each k -step expansion by alternating continuous diffusion and discrete refinement. Let $\mathcal{G}_t^{\text{latent}}$ and $\mathcal{G}_t^{\text{structure}}$ denote the latent and decoded graph states at step t . TreeDiff first performs n latent denoising steps to obtain $\mathcal{G}_{t-n}^{\text{latent}}$, then decodes it as $\mathcal{G}_{t-n}^{\text{structure}} = \text{Dec}(\mathcal{G}_{t-n}^{\text{latent}}, t - n)$. A lightweight denoiser p_ψ applies $m \ll n$ categorical refinement steps to enforce topological consistency, producing $\mathcal{G}_{t-n-m}^{\text{structure}}$, which is re-encoded as $\mathcal{G}_{t-n-m}^{\text{latent}} = \text{Enc}(\mathcal{G}_{t-n-m}^{\text{structure}}, t - n - m)$.

Rather than treating this reprojected latent as a new trajectory, TreeDiff interprets it as a directional guide that reshapes the remaining diffusion. A guidance vector is computed as

$$g_{t-n} = \nabla_{\mathcal{G}_{t-n}^{\text{latent}}} \left(-\frac{1}{2\sigma_g^2} \|\mathcal{G}_{t-n-m}^{\text{latent}} - \mathcal{G}_{t-n}^{\text{latent}}\|_2^2 \right) = \frac{\mathcal{G}_{t-n-m}^{\text{latent}} - \mathcal{G}_{t-n}^{\text{latent}}}{\sigma_g^2}, \quad (5)$$

which acts as the gradient of an implicit potential attracting the trajectory toward structure-consistent regions. For each subsequent reverse step $t' < k - n$, the guided posterior is defined as

$$p_\theta(\mathcal{G}_{t'-1}^{\text{latent}} | \mathcal{G}_{t'}^{\text{latent}}) = \mathcal{N}(\tilde{\mu}_\theta(\mathcal{G}_{t'}^{\text{latent}}, t') + h_{t'} \tilde{\beta}_{t'} g_{t-n}, \tilde{\beta}_{t'} \mathbf{I}), \quad (6)$$

where $\tilde{\mu}_\theta$ is the standard posterior mean, $\tilde{\beta}_{t'}$ is the variance schedule, and $h_{t'}$ modulates the strength of structural guidance. Intuitively, the first n latent steps explore freely under unconstrained diffusion, while the subsequent guided updates are softly attracted toward structurally valid manifolds. This allows MCTS to conduct selection and value backup entirely within the latent domain, while periodically anchoring trajectories in the discrete space to enforce validity and stability across long rollouts.

Trajectory Distillation for Auxiliary Modules. Inspired by the success of large language models in distilling high-quality reasoning trajectories, we adopt a similar principle to train the auxiliary components of TreeDiff—namely, the time-conditioned encoder $\text{Enc}(\cdot)$, decoder $\text{Dec}(\cdot)$, and discrete denoiser $p_\psi(\cdot)$. Rather than training them jointly with diffusion, we distill knowledge from pre-collected generation trajectories so that these modules capture diffusion-consistent dynamics. Concretely, we generate 500 full denoising trajectories $\{\mathcal{G}_t\}_{t=0}^T$ using a pretrained diffusion model and treat them as teacher demonstrations. The encoder-decoder pair is trained as a time-conditioned variational autoencoder that reconstructs intermediate graphs along these trajectories:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{\mathcal{G}_t} [\|\mathcal{G}_t - \text{Dec}(\text{Enc}(\mathcal{G}_t, t), t)\|_2^2] + \lambda_{\text{KL}} \text{KL}(q(z_t | \mathcal{G}_t, t) \| \mathcal{N}(0, \mathbf{I})), \quad (7)$$

which encourages temporally smooth and semantically aligned latent representations. Meanwhile, the graph-space denoiser p_ψ is trained on consecutive teacher states $(\mathcal{G}_{t+1} \rightarrow \mathcal{G}_t)$ to predict the next denoised structure:

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_t [\|\hat{\mathbf{X}}_t - \mathbf{X}_t\|_2^2 + \|\hat{\mathbf{E}}_t - \mathbf{E}_t\|_2^2], (\hat{\mathbf{X}}_t, \hat{\mathbf{E}}_t) = p_\psi(\mathcal{G}_{t+1}, t). \quad (8)$$

The architectures of these modules are implementation-agnostic.

4.3 Dual-Space Verifier

In MCTS, the *Simulation* and *Backpropagation* phases estimate long-term rewards for candidate trajectories. However, full rollouts are

prohibitively expensive for diffusion with hundreds of steps and multiple objectives. TreeDiff therefore introduces a learned *dual-space verifier* V_ϕ to predict terminal rewards from partially denoised states. At step t , it takes both the latent representation $\mathcal{G}_t^{\text{latent}}$ and the decoded structure $\mathcal{G}_t^{\text{structure}}$ as input and outputs $\hat{R}_t \approx R(\mathcal{G}_0)$. This replaces explicit rollouts during search, enabling efficient multi-objective planning while remaining consistent with dual-space denoising.

The verifier is trained as a regression model on full denoising trajectories $\{\mathcal{G}_t\}_{t=0}^T$ generated by a pretrained diffusion model, each labeled with the terminal reward $R(\mathcal{G}_0) = \sum_k w_k f_k(\mathcal{G}_0)$. To improve robustness, we apply stochastic augmentation by perturbing latent states $\tilde{\mathcal{G}}_t^{\text{latent}} = \mathcal{G}_t^{\text{latent}} + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_a^2 \mathbf{I})$, and decoding them into $\tilde{\mathcal{G}}_t^{\text{structure}}$. The verifier minimizes

$$\mathcal{L}_{\text{verifier}} = \mathbb{E}_t [\|V_\phi(\tilde{\mathcal{G}}_t^{\text{latent}}, \tilde{\mathcal{G}}_t^{\text{structure}}, t) - R(\mathcal{G}_0)\|_2^2]. \quad (9)$$

In practice, V_ϕ adopts a dual-branch architecture: a graph transformer encodes $\mathcal{G}_t^{\text{structure}}$, an MLP processes $\mathcal{G}_t^{\text{latent}}$, and cross-attention fuses both before predicting $\hat{R}_t$. During MCTS, $\hat{R}_t$ is used as the node value for simulation and backpropagation, enabling efficient evaluation without rollouts.

The dual-space verifier completes TreeDiff’s search loop by providing fast, reliable value estimates. Together with macro-step expansion and dual-space denoising, it enables latent-space planning while remaining grounded in discrete graph semantics. Since $R(\mathcal{G}_0)$ can combine arbitrary objectives, the verifier naturally supports controllable multi-objective generation at inference time.

5 Experiments

5.1 Conditional Graph Generation

Conditional Generation on 2D Molecular Graphs. We evaluate conditional graph generation on three molecular benchmarks, following Liu et al. [43]: BACE (human β -secretase 1 inhibition), BBBP (blood-brain barrier permeability), and HIV (replication inhibition). Apart from molecular properties, we additionally condition on synthetic accessibility (SAS) and synthetic complexity (SCS) [10, 15]. Datasets are split into 6:2:2 train/validation/test sets [57], and we generate 10,000 molecules per task. Evaluation metrics include diversity (Div., %), fragment-based similarity (Sim., %), mean absolute error (MAE), and classification accuracy (Acc., %). Each experiment is repeated five times and averaged to mitigate randomness. We compare against a broad set of baselines: traditional methods (GraphGA [31], MARS [83], LSTM-HC [4], JTVAE-BO [32]); diffusion-based generators (DiGress [73], DiGress v2, GDSS [34], MOOD [39], GraphDiT [43]); and inference-time guidance strategies, including Best-of-N [46], Beam Search [46], TAGMol [14], and SVDD [40]. For fairness, all inference-time methods adopt GraphDiT as the backbone, with matched compute budgets. As summarized in Table 1, inference-time guidance consistently improves diffusion baselines, and TreeDiff delivers the most stable and substantial gains across all benchmarks. It achieves the best average rank (A.R. 1.9), outperforming both diffusion and inference-only competitors. In terms of distribution quality, TreeDiff avoids the trade-offs between diversity and similarity, maintaining both at high levels (e.g., Synth.&BACE: Div. 89.30 vs. 82.38 and Sim. 87.82 vs. 87.52 for Graph DiT). For controllability, TreeDiff achieves stronger condition matching, with

Table 1: Conditional graph generation results on 2D molecules across diversity (Div., %), similarity (Sim., %), mean absolute error (MAE), and accuracy (Acc., %). Boldface and underline indicate the best and second-best, and A.R. is the average ranking.

	Method	Synth. & BACE				Synth. & BBBP				Synth. & HIV				A.R.
		Div. $\uparrow$	Sim. $\uparrow$	MAE $\downarrow$	Acc. $\uparrow$	Div. $\uparrow$	Sim. $\uparrow$	MAE $\downarrow$	Acc. $\uparrow$	Div. $\uparrow$	Sim. $\uparrow$	MAE $\downarrow$	Acc. $\uparrow$	
Tradition	Graph-GA [31]	85.85	98.05	0.963	46.90	89.50	95.09	1.208	30.15	89.93	<u>96.61</u>	0.984	60.35	7.3
	MARS [83]	83.38	88.27	1.012	51.84	86.37	76.96	1.225	51.89	87.64	<u>65.17</u>	0.969	64.55	9.5
	LSTM-HC [4]	81.46	79.82	0.921	58.16	88.83	89.32	0.997	55.90	90.91	91.45	0.948	67.36	7.8
	JTVAE-BO [32]	66.82	72.81	0.992	46.28	74.58	58.21	1.162	49.58	80.55	41.73	1.236	48.50	11.8
Diffusion	DiGress [73]	88.62	69.42	2.068	50.61	90.98	68.05	2.366	65.36	<u>91.94</u>	85.62	1.922	53.35	9.1
	DiGress v2 [73]	88.12	70.27	2.337	51.13	91.07	63.36	2.269	65.31	<u>91.93</u>	84.76	1.593	53.31	9.2
	GDSS [34]	87.56	27.08	1.642	50.36	84.15	26.72	1.379	50.37	<u>78.17</u>	10.32	1.252	48.30	11.9
	MOOD [39]	<u>89.02</u>	25.87	1.885	50.62	92.73	17.15	2.028	49.03	92.80	13.61	2.314	51.06	9.8
	Graph DiT [43]	82.38	87.52	<u>0.400</u>	91.35	88.56	93.29	<u>0.355</u>	94.17	89.74	95.75	<u>0.309</u>	<u>97.77</u>	4.8
Inference-Time	Best-of-N [46]	87.29	88.50	0.480	91.01	89.29	92.41	0.420	93.80	88.54	95.61	0.360	97.01	5.8
	Beam Search [46]	86.03	89.21	0.455	91.40	88.53	92.86	0.405	94.11	87.88	95.84	0.345	97.20	5.3
	TAGMol [14]	88.64	<u>87.45</u>	0.430	<u>91.60</u>	90.45	91.43	0.372	<u>94.46</u>	89.62	95.17	0.325	97.56	4.5
	SVDD [40]	86.51	89.03	0.510	90.56	88.81	92.69	0.460	93.50	88.04	95.73	0.395	96.83	6.3
	TreeDiff (Ours)	89.30	87.82	0.392	91.83	<u>91.62</u>	<u>93.63</u>	0.342	94.81	90.41	96.90	0.302	97.90	1.9

Table 2: Conditional graph generation results on 3D molecular benchmark QM9 (ID & OOD). We report mean absolute error (MAE) on six SE(3)-invariant quantum properties: polarizability α (Bohr³), gap $\Delta\epsilon$ (meV), HOMO energies ϵ_H (meV), LUMO energies ϵ_L (meV), dipole moment μ (D), and heat capacity C_v (cal/mol · K).

	Method	In-Distribution (MAE $\downarrow$)						Out-Of-Distribution (MAE $\downarrow$)						A.R.
		α (Bohr ³)	$\Delta\epsilon$ (meV)	ϵ_H (meV)	ϵ_L (meV)	μ (D)	C_v ($\frac{\text{cal}}{\text{mol}}\text{K}$)	α (Bohr ³)	$\Delta\epsilon$ (meV)	ϵ_H (meV)	ϵ_L (meV)	μ (D)	C_v ($\frac{\text{cal}}{\text{mol}}\text{K}$)	
Diff.	Random	41.00	193.36	103.30	121.83	8.40	13.56	73.03	<u>344.43</u>	183.00	217.01	14.96	24.15	6.3
	EDM[81]	20.15	287.00	158.70	166.20	7.01	13.63	55.70	1561.90	1196.80	228.20	19.13	38.42	7.6
	LDM-3DG [91]	15.56	107.14	54.62	63.08	6.33	13.66	32.06	363.13	109.30	178.69	22.18	31.12	4.9
Inference-Time	Best-of-N[46]	16.23	109.97	58.11	67.08	6.27	13.54	31.07	359.91	110.13	175.11	14.59	24.07	4.2
	Beam Search[46]	16.04	108.06	57.14	66.05	6.29	13.58	33.47	345.11	103.23	<u>171.03</u>	<u>14.31</u>	<u>23.79</u>	3.7
	TAGMol[14]	15.33	100.19	55.09	63.06	6.13	13.47	31.01	345.08	102.09	171.07	14.77	24.29	2.7
	SVDD[40]	16.78	115.11	<u>60.07</u>	<u>67.39</u>	<u>6.26</u>	<u>13.57</u>	34.47	370.13	115.76	178.11	14.83	23.91	5.2
	TreeDiff (Ours)	14.92	95.21	61.55	60.19	6.07	13.41	28.55	341.76	99.53	170.11	14.24	23.51	1.4

lower MAE and higher Accuracy across tasks (e.g., Synth.&HIV: MAE 0.302 and Acc. 97.90%).

Conditional Generation on 3D Molecular Graphs. We further evaluate on the 3D QM9 benchmark [27, 59], where each molecule is annotated with six SE(3)-invariant quantum properties: polarizability α (Bohr³), HOMO–LUMO gap $\Delta\epsilon$ (meV), HOMO energy ϵ_H (meV), LUMO energy ϵ_L (meV), dipole moment μ (D), and heat capacity C_v (cal/mol·K). These properties are used as conditional targets, and we measure performance by the MAE between the requested conditions and oracle-predicted values. Two evaluation settings are considered: in-distribution (ID), where conditions are sampled within the training range, and out-of-distribution (OOD), where targets are shifted beyond it. Baselines include a random sampler [91], 3D diffusion models (EDM [81], LDM-3DG [91]), and inference-time guidance methods (Best-of-N [46], Beam Search [46], TAGMol [14], and SVDD [40]), all built on the LDM-3DG backbone under matched compute. On QM9 (Table 2), TreeDiff achieves the best overall performance across both ID and OOD regimes, with the lowest average rank (A.R. 1.4). In ID, it reduces error on key properties such as polarizability and the HOMO–LUMO gap, while

also improving on ϵ_L , μ , and C_v . Under distribution shift, the advantage becomes more pronounced and consistent across all six targets. We attribute this robustness to long-horizon planning, which aligns intermediate denoising states with global property targets, and step-wise verification, which filters unstable edits and prevents error accumulation. Together, these mechanisms enable reliable conditional generation under both ID and OOD conditions.

5.2 Unconditional Graph Generation

Unconditional Generation on 2D Molecular Graphs. To assess model performance without explicit conditional supervision in training, we evaluate unconditional molecular graph generation on QM9 [59] and ZINC250k [29]. Following standard practice, we report Validity (chemical sanity without rule-based repair), distributional fidelity via Fréchet ChemNet Distance (FCD) [58], global structure mismatch via the NSPDK kernel distance [11], and Scaffold similarity. These metrics also serve as guidance signals for inference-time baselines. We compare against traditional methods (MoFlow [92], GraphAF [62], GraphDF [45]), diffusion-based generators (EDP-GNN [52], GDSS [34], DiGress [72], GruM [33]), and

Table 3: Unconditional graph generation results on 2D QM9 and ZINC250k regarding validity (Valid), Fréchet ChemNet Distance (FCD), NSPDK kernel distance (NSPDK), and scaffold similarity (Scaf.).

	Method	QM9				ZINC250k				A.R.
		Valid $\uparrow$	FCD $\downarrow$	NSPDK $\downarrow$	Scaf. $\uparrow$	Valid $\uparrow$	FCD $\downarrow$	NSPDK $\downarrow$	Scaf. $\uparrow$	
	Training set	100.00	0.040	0.0001	0.972	100.00	0.062	0.0001	0.840	–
Trad.	MoFlow [92]	91.36	4.467	0.0169	0.145	63.11	20.931	0.0455	0.013	10.4
	GraphAF [62]	74.43	5.625	0.0207	0.305	68.47	16.023	0.0442	0.067	10.0
	GraphDF [45]	93.88	10.928	0.0636	0.098	90.61	33.546	0.1770	0.000	11.0
Diffusion	EDP-GNN [52]	47.52	2.680	0.0046	0.327	87.21	76.737	0.0485	0.000	10.3
	GDSS [34]	95.72	2.900	0.0038	0.698	97.44	14.656	0.0021	0.047	8.0
	DiGress [72]	98.19	0.095	0.0033	0.935	94.99	3.482	0.0021	0.416	5.8
	GruM [33]	99.69	0.108	0.0002	0.945	98.95	2.257	0.0015	0.530	2.0
Inference-Time	Best-of-N [46]	99.22	0.153	0.0003	0.912	98.84	3.803	0.0013	0.485	4.8
	Beam Search [46]	99.81	0.123	0.0004	0.918	98.12	2.553	0.0015	0.502	3.5
	TAGMol [14]	99.63	0.142	0.0004	0.918	98.73	2.647	0.0012	0.488	3.8
	SVDD [40]	99.41	0.205	0.0006	0.905	98.42	2.706	0.0018	0.418	5.9
	TreeDiff (Ours)	99.89	0.091	0.0004	0.932	98.95	2.135	0.0011	0.492	1.8

Table 4: Unconditional molecular graph generation results on 3D QM9 and Drugs regarding validity & uniqueness (V&U), atom-level stability (AS), and molecule-level stability (MS).

	Method	QM9			Drugs	
		V&U $\uparrow$	AS $\uparrow$	MS $\uparrow$	AS $\uparrow$	A.R.
Trad.	ENF [18]	39.4	85.0	4.9	–	16.7
	G-SchNet [19]	80.3	95.7	68.1	–	15.3
Diffusion	GDM [27]	–	97.0	63.2	75.0	15.3
	GDM-Aug [27]	89.5	97.6	71.6	77.7	12.5
	EDM [81]	90.7	98.7	82.6	81.3	8.0
	EDM-Bridge [81]	90.7	98.8	84.6	82.4	6.8
	GCDM [48]	93.3	98.7	85.7	89.0	3.8
	MiDi [74]	95.0	97.9	84.0	82.5	7.0
	GraphLDM [86]	82.7	97.2	70.5	76.2	14.0
	GraphLDM-Aug [86]	89.5	97.9	78.7	79.6	11.2
	GeoLDM [86]	92.3	98.2	80.7	79.3	10.2
Inference-Time	LDM-3DG [91]	95.3	97.6	87.7	80.51	6.2
	Best-of-N [46]	93.9	98.6	87.5	85.4	4.0
	Beam Search [46]	93.2	98.5	87.1	84.6	5.5
	TAGMol [14]	92.9	98.7	85.5	85.1	5.2
	SVDD [40]	92.8	98.4	86.3	84.2	6.8
	TreeDiff (Ours)	94.8	98.8	88.6	<u>86.5</u>	1.8

inference-time guidance strategies (Best-of-N [46], Beam Search [46], TAGMol [14], SVDD [40]). We also report the training set as a reference upper bound for distributional metrics. As shown in Table 3, TreeDiff achieves the lowest FCD on both datasets and near-perfect validity, while remaining competitive on NSPDK and scaffold similarity. Compared with both diffusion backbones and inference-time guidance methods, TreeDiff provides a more reliable balance between distributional fidelity and chemical plausibility. These results highlight the effectiveness of planning-guided sampling in advancing unconditional molecular graph generation.

Unconditional Generation on 3D Molecular Graphs. We further evaluate TreeDiff on unconditional 3D molecular generation

using QM9 [59] and Drugs [2] as benchmarks. Following established protocols [27, 48, 74, 81], we report validity & uniqueness (V&U), atom-level stability (AS), and molecule-level stability (MS) on QM9, and AS on Drugs. Baselines span a broad range of 3D generators, including traditional methods (ENF [18], G-SchNet [19]); diffusion-based models (GDM [27], GDM-Aug, EDM [81], EDM-Bridge, GCDM [48], MiDi [74], GraphLDM [86], GraphLDM-Aug, GeoLDM, LDM-3DG [91]); and inference-time strategies (Best-of-N [46], Beam Search, TAGMol [14], SVDD [40]). As shown in Table 4, TreeDiff achieves the best overall performance on QM9 and the second-best AS on Drugs, yielding the lowest overall average rank (A.R. 1.8) across benchmarks. On QM9, it attains the highest molecule-level stability (MS 88.6) and state-of-the-art atom stability tied with EDM-Bridge (AS 98.8) while maintaining strong validity and uniqueness (94.8). Compared with diffusion backbones (e.g., LDM-3DG, GCDM), TreeDiff substantially reduces whole-molecule failures through global planning. Against inference-time methods, it preserves their high atom-level reliability but produces more globally consistent structures. On Drugs, TreeDiff maintains strong atom stability (86.5), confirming its robustness on larger, drug-like molecules. Overall, these results highlight TreeDiff’s ability to couple local atomic precision with global structural coherence in 3D molecular generation.

5.3 Ablation Study

Ablation on dual-space denoising. We investigate the impact of dual-space denoising strength by varying the guidance scale $\sigma \in \{0.1, 0.5, 1.0, 10\}$ on 2D and 3D QM9 datasets under unconditional settings. A *w/o graph space* variant is also included as a latent diffusion-only baseline. As shown in Table 5, excessively strong guidance ($\sigma=0.1$) over-constrains the denoising trajectory, leading to degraded validity and fidelity, whereas overly weak guidance ($\sigma=10$) approaches standard diffusion and offers limited improvement. The default configuration ($\sigma=1.0$) achieves the best overall performance across both 2D and 3D metrics, demonstrating that

Table 5: Ablation results of the dual-space denoising on 2D/3D unconditional generation tasks, where σ is used to control the impact of the dual-space denoising, where a large value indicate less impact.

Variant	2D QM9			3D QM9		
	Valid $\uparrow$	FCD $\downarrow$	NSPDK $\downarrow$	V&U $\uparrow$	AS $\uparrow$	MS $\uparrow$
TreeDiff ($\sigma = 1.0$)	99.90	0.090	0.00040	94.8	98.8	88.6
$\sigma = 0.1$	97.56	0.256	0.00100	92.5	98.6	86.9
$\sigma = 0.5$	99.48	0.112	0.00055	93.4	98.7	87.4
$\sigma = 10$	99.15	0.125	0.00064	92.4	98.5	86.8
w/o graph space	98.13	0.156	0.00087	92.1	97.2	86.8

Table 6: Ablation results on the verifier on 2D/3D conditional generation, where Latent + GPM is the default.

Verifier Type	BBBP		QM9 (ID)		QM9 (OOD)	
	Acc $\uparrow$	MAE $\downarrow$	α $\downarrow$	ϵ_H $\downarrow$	α $\downarrow$	ϵ_H $\downarrow$
Latent-only	92.4	0.405	17.2	69.5	36.0	116.2
Graph-only (GIN [84])	92.8	0.395	16.6	68.3	34.8	113.4
Graph-only (GT [60])	93.3	0.380	15.8	64.5	32.5	106.5
Graph-only (GPM [76])	93.8	0.381	15.6	65.3	33.6	104.1
Latent + GIN [84]	93.8	0.374	16.1	66.8	33.5	110.7
Latent + GT [60]	94.5	0.350	15.3	63.0	31.0	103.5
Latent + GPM [76]	94.8	0.342	14.9	61.6	28.6	99.5

moderate dual-space coupling substantially enhances generation quality compared to the diffusion-only baseline.

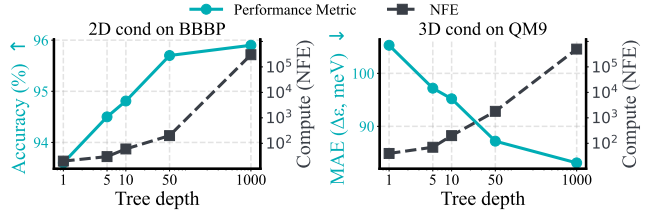
Ablation on the verifier. We examine the impact of verifier design under identical computational budgets on conditional generation tasks across BBBP, QM9 (ID), and QM9 (OOD). We consider three categories of verifiers: *Latent-only*, *Graph-only* (GIN, Graph Transformer (GT), and GPM), and *Latent+Graph* (Latent+GIN, Latent+GT, Latent+GPM). As summarized in Table 6, the Latent-only verifier provides fast feedback but lacks discrete structural cues, resulting in weaker calibration. Graph-only variants better capture structural constraints: GIN leverages local substructure sensitivity, while GT reduces prediction errors through long-range aggregation. Combining latent and graph modalities consistently improves performance—both *Latent+GIN* and *Latent+GT* surpass their single-modality counterparts. Our default *Latent+GPM* achieves the strongest results overall, indicating that global attention enhanced by graph-structural priors yields more accurate and stable node-wise calibration across noise scales.

5.4 Efficiency Analysis

Efficiency analysis on the verifier. We compare *Post-hoc* verification, which evaluates only the final denoising step ($t=0$), against two baselines—*Post-hoc + Oracle Verify* and *Post-hoc + TreeDiff Verify*—as well as our proposed *Step-wise + TreeDiff Verify*. As shown in Table 7, step-wise verification achieves the best overall balance between accuracy and efficiency, reducing computation cost by over 9 $\times$ on BACE and 15 $\times$ on QM9 (ID) while maintaining comparable or even slightly better predictive accuracy. In contrast, post-hoc

Table 7: Efficiency analysis on the verifier for 2D/3D conditional generation.

Variant	BACE			QM9 (ID)		
	Acc $\uparrow$	MAE $\downarrow$	Comp.	ϵ_L $\downarrow$	C_v $\downarrow$	Comp.
Post-hoc + TreeDiff Verify	87.1	0.409	9.6x	59.5	13.0	15.6x
Post-hoc + Oracle Verify	86.7	0.420	8.9x	60.0	13.5	14.2x
Step-wise + TreeDiff Verify	87.8	0.392	1.0x	60.2	13.4	1.0x

**Figure 4: Efficiency analysis on the MCTS tree depth vs. inference-time computation.**

variants require substantially higher compute since verification occurs only after full trajectory generation.

Efficiency analysis on tree depth. We vary the macro-step budget (tree depth) $D \in \{1, 5, 10, 50, 1000\}$ and report task performance alongside inference compute (NFE) for two conditional generation settings: 2D BBBP and 3D QM9 (ID). As reported in Figure 4, model performance improves rapidly for small depths between $D=1$ and $D=10$ then plateaus beyond $D \geq 50$. Meanwhile, NFE grows near-geometrically with D due to repeated expansions and evaluations, revealing a clear compute-quality trade-off. A practical Pareto regime emerges at $D \in [10, 50]$, which captures most quality gains while keeping computation moderate.

6 Conclusion

We propose TreeDiff, an inference-time framework that couples MCTS with dual-space diffusion to enable controllable and stable graph generation without retraining. The architecture comprises three key components: (i) *macro-step expansion* that operates over groups of denoising steps to reduce search depth, (ii) *dual-space denoising* that jointly leverages latent efficiency and graph-level structural fidelity, and (iii) a *dual-space verifier* that provides rollout-free node-wise evaluation for early pruning. Extensive experiments on 2D/3D molecular generation, across unconditional and conditional regimes and ID/OOD settings, demonstrate the state-of-the-art performance and favorable inference-time scaling of TreeDiff.

Acknowledgments

The work of Z. Wang, C. Zhang, and Y. Ye was partially supported by the NSF under grants IIS-2533550, IIS-2321504, IIS-2217239, CNS-2426514, and CMMI-2146076, Notre Dame Strategic Framework Research Grant (2025), and Notre Dame Poverty Research Package (2025). Any expressed opinions, findings, and conclusions or recommendations are those of the authors and do not necessarily reflect the views of the sponsors.

References

- [1] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine learning* (2002).
- [2] Simon Axelrod and Rafael Gomez-Bombarelli. 2022. GEOM, energy-annotated molecular conformations for property prediction and molecular generation. *Scientific Data* (2022).
- [3] Angela Bonifati, Irena Holubová, Arnau Prat-Pérez, and Sherif Sakr. 2020. Graph generators: State of the art and open challenges. *CSUR* (2020).
- [4] Nathan Brown, Marco Fiscato, Marwin HS Segler, and Alain C Vaucher. 2019. GuacaMol: benchmarking models for de novo molecular design. *Journal of chemical information and modeling* (2019).
- [5] Gabriel Cardoso, Yazid Janati El Idrissi, Sylvain Le Corff, and Eric Moulines. 2023. Monte Carlo guided diffusion for Bayesian linear inverse problems. *ICLR* (2023).
- [6] Xiaohui Chen, Yukun Li, Aonan Zhang, and Li-ping Liu. 2022. Nvdiffr: Graph generation through the diffusion of node vectors. *arXiv preprint arXiv:2211.10794* (2022).
- [7] Zhe Chen, Yuehan Wang, Bin Zhao, Jing Cheng, Xin Zhao, and Zongtao Duan. 2020. Knowledge graph completion: A review. *Ieee Access* (2020).
- [8] Hyuna Cho, Minjae Jeong, Sooyeon Jeon, Sungsoo Ahn, and Won Hwa Kim. 2023. Multi-resolution spectral coherence for graph generation with score-based diffusion. *neurIPS* (2023).
- [9] Hyungjin Chung, Jeongsol Kim, Michael T Mccann, Marc L Klasky, and Jong Chul Ye. 2022. Diffusion posterior sampling for general noisy inverse problems. *ICLR* (2022).
- [10] Connor W Coley, Luke Rogers, William H Green, and Klavs F Jensen. 2018. SCScore: synthetic complexity learned from a reaction corpus. *Journal of chemical information and modeling* (2018).
- [11] Fabrizio Costa and Kurt De Grave. 2010. Fast neighborhood subgraph pairwise distance kernel. In *ICML*.
- [12] Nicola De Cao and Thomas Kipf. 2018. MolGAN: An implicit generative model for small molecular graphs. *ICML* (2018).
- [13] Prafulla Dhariwal and Alexander Nichol. 2021. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems* (2021).
- [14] Vineeth Dorna, D Subhalingam, Keshav Kolluru, Shreshth Tuli, Mrityunjay Singh, Saurabh Singal, NM Krishnan, and Sayan Ranu. 2024. Tagmol: Target-aware gradient-guided molecule generation. *ICML* (2024).
- [15] Peter Ertl and Ansgar Schuffenhauer. 2009. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of cheminformatics* (2009).
- [16] Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. 2023. Alphazero-like tree-search can guide large language model decoding and training. *ICML* (2023).
- [17] Fabian Fuchs, Daniel Worrall, Volker Fischer, and Max Welling. 2020. Se (3)-transformers: 3d roto-translation equivariant attention networks. *neurIPS* (2020).
- [18] Victor Garcia Satorras, Emiel Hoogeboom, Fabian Fuchs, Ingmar Posner, and Max Welling. 2021. E (n) equivariant normalizing flows. *neurIPS* (2021).
- [19] Niklas Gebauer, Michael Gastegger, and Kristof Schütt. 2019. Symmetry-adapted generation of 3d point sets for the targeted discovery of molecules. *neurIPS* (2019).
- [20] Shanshan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and LingPeng Kong. 2022. Diffuseq: Sequence to sequence text generation with diffusion models. *ICLR* (2022).
- [21] Jiaqi Guan, Wesley Wei Qian, Xingang Peng, Yufeng Su, Jian Peng, and Jianzhu Ma. 2023. 3d equivariant diffusion for target-aware molecule generation and affinity prediction. *ICLR* (2023).
- [22] Jiaqi Guan, Xiangxin Zhou, Yuwei Yang, Yu Bao, Jian Peng, Jianzhu Ma, Qiang Liu, Liang Wang, and Quanquan Gu. 2024. DecompDiff: diffusion models with decomposed priors for structure-based drug design. *ICML* (2024).
- [23] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *neurIPS* (2017).
- [24] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. *ACL* (2023).
- [25] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *neurIPS* (2020).
- [26] Jonathan Ho and Tim Salimans. 2022. Classifier-free diffusion guidance. *neurIPS* (2022).
- [27] Emiel Hoogeboom, Victor Garcia Satorras, Clément Vignac, and Max Welling. 2022. Equivariant diffusion for molecule generation in 3d. In *ICML*.
- [28] Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Mohammadamin Barekattain, Simon Schmitt, and David Silver. 2021. Learning and planning in complex action spaces. In *ICML*.
- [29] John J Irwin, Teague Sterling, Michael M Mysinger, Erin S Bolstad, and Ryan G Coleman. 2012. ZINC: a free tool to discover chemistry for biology. *Journal of chemical information and modeling* (2012).
- [30] Vineet Jain, Kusha Sareen, Mohammad Pedramfar, and Siamak Ravanbakhsh. 2025. Diffusion Tree Sampling: Scalable inference-time alignment of diffusion models. *arXiv preprint arXiv:2506.20701* (2025).
- [31] Jan H Jensen. 2019. A graph-based genetic algorithm and generative model/Monte Carlo tree search for the exploration of chemical space. *Chemical science* (2019).
- [32] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. 2018. Junction tree variational autoencoder for molecular graph generation. In *ICML*.
- [33] Jaehyeon Jo, Dongki Kim, and Sung Ju Hwang. 2023. Graph generation with diffusion mixture. *ICML* (2023).
- [34] Jaehyeon Jo, Seul Lee, and Sung Ju Hwang. 2022. Score-based generative modeling of graphs via the system of stochastic differential equations. In *ICML*.
- [35] Mingxuan Ju, Wenhao Yu, Tong Zhao, Chuxu Zhang, and Yanfang Ye. 2022. Grape: Knowledge graph enhanced passage reader for open-domain question answering. *arXiv preprint arXiv:2210.02933* (2022).
- [36] Seiji Kajita, Tomoyuki Kinjo, and Tomoki Nishi. 2020. Autonomous molecular design by Monte-Carlo tree search and rapid evaluations using molecular dynamics simulations. *Communications Physics* (2020).
- [37] Gwanghyun Kim, Taesung Kwon, and Jong Chul Ye. 2022. Diffusionclip: Text-guided diffusion models for robust image manipulation. In *CVPR*.
- [38] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.
- [39] Seul Lee, Jaehyeon Jo, and Sung Ju Hwang. 2023. Exploring chemical space with score-based out-of-distribution generation. In *ICML*.
- [40] Xiner Li, Yulai Zhao, Chenyu Wang, Gabriele Scalia, Gokcen Eraslan, Surag Nair, Tommaso Biancalani, Shuiwang Ji, Aviv Regev, Sergey Levine, et al. 2024. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. *arXiv preprint arXiv:2408.08252* (2024).
- [41] Renjie Liao, Yujia Li, Yang Song, Shenlong Wang, Will Hamilton, David K Duvenaud, Raquel Urtasun, and Richard Zemel. 2019. Efficient graph generation with graph recurrent attention networks. *neurIPS* (2019).
- [42] Gang Liu, Eric Inae, Tong Zhao, Jiaxin Xu, Tengfei Luo, and Meng Jiang. 2023. Data-centric learning from unlabeled graphs with diffusion model. *neurIPS* (2023).
- [43] Gang Liu, Jiaxin Xu, Tengfei Luo, and Meng Jiang. 2024. Graph diffusion transformers for multi-conditional molecular generation. In *neurIPS*.
- [44] Tianze Luo, Zhanfeng Mo, and Sinno Jialin Pan. 2023. Fast graph generation via spectral diffusion. *TPAMI* (2023).
- [45] Youzhi Luo, Keqiang Yan, and Shuiwang Ji. 2021. Graphdf: A discrete flow model for molecular graph generation. In *ICML*.
- [46] Nanye Ma, Shangyuan Tong, Haolin Jia, Hexiang Hu, Yu-Chuan Su, Mingda Zhang, Xuan Yang, Yandong Li, Tommi Jaakkola, Xuhui Jia, et al. 2025. Scaling Inference Time Compute for Diffusion Models. In *CVPR*.
- [47] Krzysztof Maziarczyk, Henry Jackson-Flux, Pashmina Cameron, Finton Sirockin, Nadine Schneider, Nikolaus Stiefl, Marwin Segler, and Marc Brockschmidt. 2021. Learning to extend molecular scaffolds with structural motifs. *ICLR* (2021).
- [48] Alex Morehead and Jianlin Cheng. 2024. Geometry-complete diffusion for 3D molecule generation and optimization. *Communications Chemistry* (2024).
- [49] Seth A Myers, Aneesh Sharma, Pankaj Gupta, and Jimmy Lin. 2014. Information network or social network? The structure of the Twitter follow graph. In *WWW*.
- [50] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. 2021. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741* (2021).
- [51] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. *arXiv preprint arXiv:2502.09992* (2025).
- [52] Chenhao Niu, Yang Song, Jiaming Song, Shengjia Zhao, Aditya Grover, and Stefano Ermon. 2020. Permutation invariant graph generation via score-based generative modeling. In *AISTATS*.
- [53] Yuta Oshima, Masahiro Suzuki, Yutaka Matsuo, and Hiroki Furuta. 2025. Inference-time text-to-video alignment with diffusion latent beam search. *arXiv preprint arXiv:2501.19252* (2025).
- [54] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *ICCV*.
- [55] Angus Phillips, Hai-Dang Dau, Michael John Hutchinson, Valentin De Bortoli, George Deligiannidis, and Arnaud Doucet. 2024. Particle denoising diffusion sampler. *arXiv preprint arXiv:2402.06320* (2024).
- [56] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. 2024. SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. In *ICLR*.
- [57] Daniil Polykovskiy, Alexander Zhebrak, Benjamin Sanchez-Lengeling, Sergey Golovanov, Oktai Tatanov, Stanislav Belyaev, Rauf Kurbanov, Aleksey Artamonov, Vladimir Aladinskiy, Mark Veselov, et al. 2020. Molecular sets (MOSES): a benchmarking platform for molecular generation models. *Frontiers in pharmacology* (2020).
- [58] Kristina Preuer, Philipp Renz, Thomas Unterthiner, Sepp Hochreiter, and Gunter Klambauer. 2018. Fréchet ChemNet distance: a metric for generative models for molecules in drug discovery. *Journal of chemical information and modeling* (2018).

- [59] Raghunathan Ramakrishnan, Pavlo O Dral, Matthias Rupp, and O Anatole Von Lilienfeld. 2014. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific data* (2014).
- [60] Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a general, powerful, scalable graph transformer. *NeurIPS* (2022).
- [61] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *CVPR*.
- [62] Chence Shi, Minkai Xu, Zhaocheng Zhu, Weinan Zhang, Ming Zhang, and Jian Tang. 2020. Graphaf: a flow-based autoregressive model for molecular graph generation. *ICLR* (2020).
- [63] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* (2016).
- [64] Martin Simonovsky and Nikos Komodakis. 2018. Graphvae: Towards generation of small graphs using variational autoencoders. In *ICML*.
- [65] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*.
- [66] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2020. Score-based generative modeling through stochastic differential equations. *ICLR* (2020).
- [67] Kyle Swanson, Gary Liu, Denise B Catacutan, Autumn Arnold, James Zou, and Jonathan M Stokes. 2024. Generative AI for designing and validating easily synthesizable and structurally novel antibiotics. *NMI* (2024).
- [68] Mingyue Tang, Carl Yang, and Pan Li. 2022. Graph auto-encoder via neighborhood wasserstein reconstruction. *ICLR* (2022).
- [69] Rylee Thompson, Boris Knyazev, Elahe Ghalebi, Jungtaek Kim, and Graham W Taylor. 2022. On evaluation metrics for graph generative models. *ICLR* (2022).
- [70] Masatoshi Uehara, Yulai Zhao, Chenyu Wang, Xiner Li, Aviv Regev, Sergey Levine, and Tommaso Biancalani. 2025. Inference-time alignment in diffusion models with reward-guided generation: Tutorial and review. *arXiv preprint arXiv:2501.09685* (2025).
- [71] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR*.
- [72] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. 2022. Digress: Discrete denoising diffusion for graph generation. *ICLR* (2022).
- [73] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. 2023. DiGress: Discrete Denoising diffusion for graph generation. In *ICLR*.
- [74] Clement Vignac, Naghman Osman, Laura Toni, and Pascal Frossard. 2023. Midi: Mixed graph and 3d denoising diffusion for molecule generation. In *ECML*.
- [75] Zehong Wang, Zheyuan Zhang, Nitesh V Chawla, Chuxu Zhang, and Yanfang Ye. 2024. GFT: Graph Foundation Model with Transferable Tree Vocabulary. In *NeurIPS*.
- [76] Zehong Wang, Zheyuan Zhang, Tianyi Ma, Nitesh V Chawla, Chuxu Zhang, and Yanfang Ye. 2025. Beyond Message Passing: Neural Graph Pattern Machine. In *ICML*.
- [77] Zehong Wang, Zheyuan Zhang, Tianyi Ma, Nitesh V Chawla, Chuxu Zhang, and Yanfang Ye. 2025. Towards Graph Foundation Models: Learning Generalities Across Graphs via Task-Trees. In *ICML*.
- [78] Zehong Wang, Zheyuan Zhang, Tianyi Ma, Chuxu Zhang, and Yanfang Ye. 2025. Scalable Graph Generative Modeling via Substructure Sequences. *NeurIPS* (2025).
- [79] Zehong Wang, Zheyuan Zhang, Chuxu Zhang, and Yanfang Ye. 2024. Subgraph Pooling: Tackling Negative Transfer on Graphs. In *IJCAI*.
- [80] Joseph L Watson, David Juergens, Nathaniel R Bennett, Brian L Trippe, Jason Yim, Helen E Eisenach, Woody Ahern, Andrew J Borst, Robert J Ragotte, Lukas F Milles, et al. 2023. De novo design of protein structure and function with RFdiffusion. *Nature* (2023).
- [81] Lemeng Wu, Chengyue Gong, Xingchao Liu, Mao Ye, and Qiang Liu. 2022. Diffusion-based molecule generation with informative prior bridges. *NeurIPS* (2022).
- [82] Luhuan Wu, Brian Trippe, Christian Naesseth, David Blei, and John P Cunningham. 2023. Practical and asymptotically exact conditional sampling in diffusion models. *neurIPS* (2023).
- [83] Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. 2021. {MARS}: Markov Molecular Sampling for Multi-objective Drug Discovery. In *ICLR*.
- [84] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *ICLR*.
- [85] Minkai Xu, Shitong Luo, Yoshua Bengio, Jian Peng, and Jian Tang. 2021. Learning neural generative dynamics for molecular conformation generation. *ICLR* (2021).
- [86] Minkai Xu, Alexander S Powers, Ron O Dror, Stefano Ermon, and Jure Leskovec. 2023. Geometric latent diffusion models for 3d molecule generation. In *ICML*.
- [87] Minkai Xu, Lantao Yu, Yang Song, Chence Shi, Stefano Ermon, and Jian Tang. 2022. Geodiff: A geometric diffusion model for molecular conformation generation. *ICLR* (2022).
- [88] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. 2023. Diffusion models: A comprehensive survey of methods and applications. *CSUR* (2023).
- [89] Jaesik Yoon, Hyeonseo Cho, Doojin Baek, Yoshua Bengio, and Sungjin Ahn. 2025. Monte carlo tree diffusion for system 2 planning. *ICML* (2025).
- [90] Jiaxuan You, Rex Ying, Xiang Ren, William Hamilton, and Jure Leskovec. 2018. Graphrnn: Generating realistic graphs with deep auto-regressive models. In *ICML*.
- [91] Yuning You, Ruida Zhou, Jiwoong Park, Haotian Xu, Chao Tian, Zhangyang Wang, and Yang Shen. 2024. Latent 3d graph diffusion. *ICLR*.
- [92] Chengxi Zang and Fei Wang. 2020. Moflow: an invertible flow model for generating molecular graphs. In *KDD*.
- [93] Jianan Zhao, Qianlong Wen, Shiyu Sun, Yanfang Ye, and Chuxu Zhang. 2021. Multi-view self-supervised heterogeneous graph embedding. In *Joint European conference on machine learning and knowledge discovery in databases*. Springer, 319–334.

A Pseudo Code

TreeDiff’s inference procedure is summarized in Algorithm 1.

Algorithm 1 TreeDiff Inference

Require: Diffusion predictor ϵ_θ ; total steps T ; start step t_s ; encoder/decoder (Enc, Dec); discrete refiner p_ψ ; verifier V_ϕ ; max children K ; rollouts per round N_r ; depth budget $D_{\max}$; UCB constant c_{ucb} .

Ensure: Final graph $\mathcal{G}_0$.

- 1: **Initialize:** latent root z_{t_s} ; set node $\mathcal{R} \leftarrow z_{t_s}$ with stats $Q=0$, $N=0$; set remaining depth $D \leftarrow D_{\max}$.
 - 2: **while** $t_s > 0$ **do**
 - 3: **for** $i = 1$ to N_r **do**
 - 4: **Selection:** From node u (latent state z_t), descend by UCB until a leaf/terminal.
 - 5: **Expansion:** At node u with time t , propose up to K children via:
 - **Jump (macro-step):** sample a step length k (e.g., around $\lfloor t/D \rfloor$) and advance $z_t \rightarrow z_{t-k}$ by k reverse latent updates.
 - **Dual-Space Denoising (Sec. Dual-Space Denoising):** (a) decode $\mathcal{G}_{t-k}^{\text{str}} = \text{Dec}(z_{t-k}, t-k)$; (b) apply lightweight discrete refinement with p_ψ ; (c) re-encode to obtain a structure-aware latent $\tilde{z}_{t-k}$ to anchor the child state.

The resulting child stores both latent $\tilde{z}_{t-k}$ and its decoded graph $\tilde{\mathcal{G}}_{t-k}^{\text{str}}$.
 - 6: **Simulation (Sec. Dual-Space Verifier):** For each child, score with $V_\phi(\tilde{z}_{t-k}, \tilde{\mathcal{G}}_{t-k}^{\text{str}}, t-k)$; initialize $Q(\cdot)$ and $N(\cdot)$ accordingly.
 - 7: **Backpropagation:** Propagate the child score along the selected path (update visits and values).
 - 8: **end for**
 - 9: **Commit:** Select the best child of the root by average value; set it as new root $\mathcal{R}$. Update $t_s \leftarrow t_s - k(\mathcal{R})$, $D \leftarrow \max(D-1, 1)$. (Optionally prune siblings to keep top- M .)
 - 10: **end while**
 - 11: **Return:** $\mathcal{G}_0 \leftarrow \text{Dec}(z_0, 0)$.
-

Procedure. Starting from a latent root z_{t_s} , TreeDiff performs a sequence of short MCTS rounds to construct a single high-quality denoising path. In each round, **Selection** first descends from the

current root to a leaf via UCB, focusing computation on promising branches. Given the selected leaf at time t , **Expansion** then proposes up to K children in two phases: a macro-step jump that advances k reverse updates in latent space, followed by *dual-space denoising*—decode to graph space, apply lightweight discrete refinement, and re-encode—to anchor a structure-aware latent state. Next, **Simulation** evaluates each child using the dual-space verifier on both the latent and its decoded graph, avoiding costly full rollouts while preserving structural feedback. Finally, **Backpropagation** updates visit counts and value estimates along the traversed path. After accumulating N_r rollouts, TreeDiff *commits* the best child as the new root, reduces t by k , and repeats until $t=0$; the resulting z_0 is then decoded to $\mathcal{G}_0$.

B Time Complexity

Let C_{lat} be the cost of one latent reverse step, C_{ref} the cost of one dual-space refinement (decode + discrete refine + re-encode), and C_{ver} the cost of one verifier call. At each visited node, UCB-based **Selection** scans its children; if the branching factor is bounded by K , the path cost is $O(K D_{\text{max}})$ (or $O(D_{\text{max}})$ when child stats are cached and K is treated as a constant).

For a leaf that is expanded into up to K children, the per-child cost of **Expansion+Simulation** is

$$O(k C_{\text{lat}} + C_{\text{ref}} + C_{\text{ver}}),$$

where k is the sampled macro-step length for that child. Therefore, one MCTS *round* with N_r rollouts incurs

$$O\left(N_r [K D_{\text{max}} + K(k C_{\text{lat}} + C_{\text{ref}} + C_{\text{ver}})]\right).$$

Across the whole inference, the number of commits is roughly $T/\bar{k}$ (with $\bar{k}$ the average macro-step), and never exceeds D_{max} . Using $\bar{k}$ to summarize k 's variability, the total work is

$$O\left(\frac{T}{\bar{k}} N_r [K D_{\text{max}} + K(\bar{k} C_{\text{lat}} + C_{\text{ref}} + C_{\text{ver}})]\right).$$

If K is constant and selection bookkeeping is $O(1)$ per level, this simplifies to

$$O\left(\frac{T}{\bar{k}} N_r [D_{\text{max}} + \bar{k} C_{\text{lat}} + C_{\text{ref}} + C_{\text{ver}}]\right).$$

Discussion.(i) **Trade-off via $\bar{k}$.** Larger $\bar{k}$ reduces the number of commit steps ($T/\bar{k}$) but increases the per-child expansion cost ($\bar{k} C_{\text{lat}}$); small $\bar{k}$ does the opposite. (ii) **Verifier amortization.** When V_ϕ is a lightweight MLP, $C_{\text{ver}} \ll \bar{k} C_{\text{lat}}$, so runtime is dominated by latent steps and occasional refinement. If V_ϕ batches B children on accelerator, its *amortized* cost becomes C_{ver}/B . (iii) **Parallelism.** The K children of a leaf and the N_r rollouts within a round are embarrassingly parallel; with P workers the effective factor becomes K/P or N_r/P in the bracketed term. (iv) **Degenerate cases.** With $K=1$ and $N_r=1$, TreeDiff reduces to a single guided trajectory with cost $O(T C_{\text{lat}} + \frac{T}{\bar{k}} (C_{\text{ref}} + C_{\text{ver}}))$. If dual-space refinement is skipped ($C_{\text{ref}}=0$), complexity coincides with pure latent MCTS. (v) **Memory.** Active-node storage is $O(N_r K)$ per round without pruning, or $O(M D_{\text{max}})$ when keeping only top- M siblings per level.

C Implementation Details

All diffusion backbones are pretrained under standard noise schedules before integration into TreeDiff. For macro-step expansion, we set the tree depth to $D=10$ and expansion width to $B=4$, with an adaptive expansion variance $\sigma_k=0.1$, and use PUCT as the selection policy. For dual-space denoising, the guidance scale is $\sigma_g=1.0$ and $h_{t'}=1.0$; the latent denoising horizon is set to half of the total diffusion steps within each macro step ($n=k/2$), while the structural refinement length is set to 10% of that value ($m=k/10$). The encoder, decoder, and verifier are trained through trajectory distillation and remain frozen during search. For 2D molecular generation, we employ a Graph Transformer [60] as the encoder-decoder backbone, and for 3D molecular generation, we use a SE(3)-Transformer [17]. The verifier adopts the recently proposed GPM [76], which excels at capturing substructural dependencies in graphs.